

Relatório técnico - Databases for Data Science

Instituição:	Faculdade UniFECAF
Disciplina:	Databases For Data Science
Aluno:	Antônio Benevaldo Chaves Santana
Professor:	Felipe Becker Nunes
Curso:	Ciência de dados

1. Resumo executivo e objetivos

Decisão central. Adotar PostgreSQL como fonte transacional, MongoDB para eventos digitais e um lakehouse em Parquet/Delta como histórico governado e camada analítica. A separação reduz o conflito entre cargas operacionais, ingestão flexível e consultas de grande volume.

A VarejoMix precisa consolidar e-commerce e lojas físicas para produzir painéis diários confiáveis de receita, margem, ruptura, conversão e cohorts, além de um dataset reproduzível para propensão de recompra e segmentação. O cenário estimado - 2 a 5 milhões de pedidos e 200 a 400 milhões de eventos ao ano - inviabiliza manter todas as necessidades em um único modelo improvisado. Consultas analíticas sobre o OLTP competiriam com transações, enquanto documentos de navegação sem contrato aumentariam duplicidade e custo.

1.1 Objetivos do desenho

OBJETIVO	CRITÉRIO DE ACEITAÇÃO
Confiabilidade	Chaves e restrições no OLTP; validação de schema; reconciliação entre fontes e Gold.
Reprocessamento	Pipelines idempotentes por período, run_id e MERGE; versão do dataset registrada.
Performance	Índices guiados por EXPLAIN; leitura colunar; separação do consumo analítico.
Ciência de Dados	Extração SQLAlchemy/pandas, corte temporal, features documentadas e limites do snapshot.
Custo e operação	Componentes open-source, processamento diário e adoção incremental por um time pequeno.

1.2 Escopo e premissas

O recorte prático implementa DDL/DML PostgreSQL, views analíticas, consultas com CTE e janela, índices/EXPLAIN, contrato e carga MongoDB e notebook com exportação CSV/Parquet. A arquitetura Bronze/Silver/Gold, as tabelas dimensionais físicas, o agendamento diário, CDC e Delta Lake são propostas de evolução, não serviços implementados neste pacote.

O projeto prepara a base para recompra e segmentação RFM, sem treinar modelo preditivo. A execução local de 20/09/2026 às 20:39 UTC (17:39 em UTC-3), com PostgreSQL 16.15 e MongoDB 7, concluiu SQL, reconciliação, planos MongoDB e exportação. Os 11 testes de regressão passaram. A falha anterior do coletor foi corrigida e sua execução confirmada. Os testes PGLite permanecem separados em evidencias/revisao.

2. Arquitetura proposta de ponta a ponta

Na arquitetura proposta, o PostgreSQL preserva a operação, o MongoDB recebe eventos e o lakehouse concentra histórico e consumo. Na implementação mínima, views no próprio PostgreSQL sustentam as consultas e a extração Python. A separação física do consumo e a ingestão diária representadas no diagrama são etapas futuras.

Arquitetura proposta e fluxo de dados

ingestão → transformação → consumo analítico

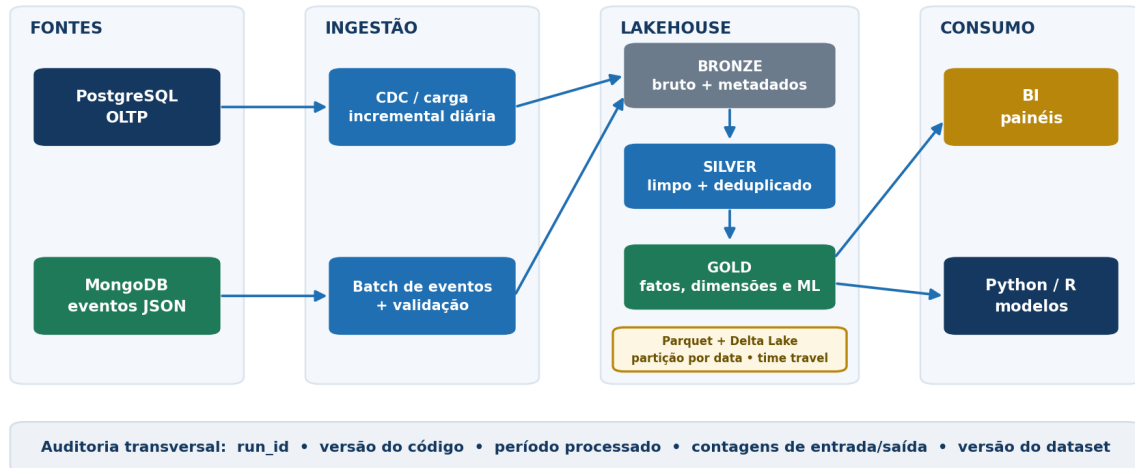


Figura 1 - Arquitetura lógica e fluxo proposto para a VarejoMix. Fonte: elaboração própria (2026).

CAMADA	PAPEL	TECNOLOGIA / DECISÃO
OLTP	Pedidos, itens, pagamentos, estoque e cadastros	PostgreSQL; transações ACID, PK/FK, CHECK e unicidade.
Eventos	Telemetria de site/app em documentos	MongoDB; um documento por evento, schema versionado e índices seletivos.
Ingestão	Incrementos diários e posterior CDC	Python/orquestrador simples; Kafka somente quando latência/volume justificarem.
Lakehouse	Histórico, qualidade, reprocessamento e auditoria	Object storage + Parquet/Delta; Bronze, Silver e Gold.
Consumo	BI, SQL ad hoc e datasets de ML	Views/tabelas Gold, DuckDB/engine SQL, Python ou R.

A fronteira mais importante é a propriedade do dado: estoque, pagamento e status de pedido continuam transacionais; eventos são observações imutáveis; métricas e features são produtos derivados e versionados. Esse contrato evita que divergências sejam “corrigidas” manualmente apenas no painel.

3. Fluxo proposto e controles implementados

3.1 Ingestão até o consumo - evolução proposta

ETAPA	ENTRADA → SAÍDA	CONTROLE
1. Captura	PostgreSQL/MongoDB → Bronze	Watermark ou offset, schema_version, ingestion_ts e run_id.
2. Padronização	Bronze → Silver	Tipos, UTC, deduplicação, regras de domínio e quarentena.
3. Reconciliação	Silver → Gold	Pedidos válidos, itens, pagamentos e receita reconciliados.
4. Publicação	Gold → BI / notebook	Testes aprovados, catálogo atualizado e versão fixada.
5. Reexecutar	Período + versão de código → nova versão	MERGE idempotente; contagens antes/depois; rollback por time travel.

3.2 Zonas e formatos

Bronze retém o payload fiel e metadados técnicos; Silver aplica qualidade e uma visão consistente das entidades; Gold materializa fatos, dimensões e datasets. Parquet organiza dados por coluna, permitindo ler apenas colunas necessárias e aproveitar compressão/encoding em grandes varreduras (APACHE PARQUET, s.d.). Delta Lake adiciona transações ACID, schema enforcement, MERGE e time travel sobre o data lake (DELTA LAKE, s.d.).

Particionamento inicial. Usar ingestion_date na Bronze e order_date/event_date nas tabelas grandes Silver/Gold. Não particionar por customer_id, product_id ou session_id: a cardinalidade geraria muitos diretórios e arquivos pequenos. Tabelas pequenas e dimensões permanecem sem partição.

3.3 Auditoria e qualidade

Implementado no VarejoMix: cada tentativa de exportação iniciada pelo pipeline registra run_id, horário, período, versão do código e estado em audit.pipeline_runs. Exportações concluídas preservam CSV, Parquet, dicionário e manifesto com hashes. A tabela começa vazia, sem registros de auditoria simulados. No lakehouse proposto, a mesma trilha deverá registrar também a versão Delta e os controles de publicação.

O script sql/06_validacoes.sql verifica cronologia, pagamentos, totais, estoque e grão analítico, além de testar que o custo atual do cadastro não altera margens históricas. A execução diária agendada, MERGE e time travel permanecem no desenho futuro; os snapshots de arquivos do recorte não são tabelas Delta.

4. Modelagem relacional OLTP

O modelo separa clientes, lojas, produtos, pedidos, itens, pagamentos e snapshots de estoque, com PK, FK, CHECK e unicidade. O total do pedido é reconciliado com os itens. Cada item guarda `unit_price` e `unit_cost_at_sale`, preservando preço e custo históricos mesmo quando o cadastro do produto é alterado.

RELAÇÃO / REGRA	IMPLEMENTAÇÃO	RISCO EVITADO
Cliente 1:N pedido	<code>orders.customer_id</code> → <code>customers.customer_id</code>	Pedido sem cliente ou identificação inconsistente.
Pedido 1:N item	<code>UNIQUE(order_id, line_number)</code> + <code>ON DELETE CASCADE</code>	Linha duplicada e item órfão.
Produto 1:N item	<code>FK product_id</code> ; preço e custo históricos no item	Mudança no preço atual alterando venda passada.
Canal e loja	<code>CHECK</code> : loja obrigatória no canal <code>STORE</code> e nula no <code>ECOMMERCE</code>	Combinações operacionais inválidas.
Pagamento	<code>FK</code> , referência única e domínios de método/status	Duplicidade de transação e valores negativos.
Estoque diário	<code>PK(snapshot_date, store_id, product_id)</code>	Snapshots duplicados e ruptura impossível de auditar.

4.1 Normalização versus desnormalização

A normalização reduz anomalias no OLTP. O preço e o custo praticados são fatos históricos do item, não cópias indevidas do preço e custo atuais do produto. Para análise, a view `vw_sales_line` expõe o grão item; o modelo estrela com fatos e dimensões físicas é uma proposta de desnormalização para a futura Gold.

4.2 Consistência e disponibilidade

Na confirmação de compra, consistência é mais importante que disponibilidade irrestrita: pedido, itens, pagamento e baixa/reserva de estoque precisam de transação ou saga controlada. Uma venda não deve aparecer paga sem pedido correspondente. Já dashboards diários toleram consistência eventual por algumas horas, desde que mostrem freshness e só publiquem lotes reconciliados.

Carga VarejoMix: 200 clientes, 40 produtos, 5 lojas, 5.000 pedidos, 10.068 itens, 5.000 pagamentos e 2.800 snapshots de estoque. Há 4.730 vendas confirmadas, totalizando R\$ 2.688.893,36. Os pedidos são gerados por MD5 com intervalo ativo nunca anterior ao cadastro. Os eventos usam semente fixa e pedidos reais dessa carga: 1.088 documentos em 500 sessões, com 78 compras reconciliáveis. Todos os dados são sintéticos.

Índices de chaves estrangeiras apoiam joins e manutenção. Índices adicionais só são criados após observar filtros e planos, pois toda estrutura acelera algumas leituras, mas consome espaço e adiciona custo às escritas.

5. Modelagem NoSQL dos eventos

A coleção `events` adota um documento por evento e um `event_id` único. Arrays de toda a sessão ou de todo o histórico do cliente não são usados, pois cresceriam sem limite e concentrariam atualizações. O contrato aceita `page_view`, `search`, `add_to_cart`, `checkout_start` e `purchase`, sempre com `event_time`, `session_id` e `schema_version`.

CRITÉRIO	EMBUTIR	REFERENCIAR	DECISÃO
Leitura	Contexto lido junto	Entidade consultada separadamente	device/page embutidos; mestres por ID.
Cardinalidade	Pequena e limitada	Alta ou muitos-para-muitos	Sessão referenciada por <code>session_id</code> .
Atualização	Mesmo ciclo de vida	Ciclo independente	Cliente/produto permanecem no OLTP.
Histórico	Fotografia do evento	Estado mestre atual	Preço exibido embutido; produto por <code>product_id</code> .
Tamanho	Previsível	Pode crescer sem limite	Nunca embutir lista infinita de eventos.

5.1 Exemplo de documento

```
{ "event_id": "evt-000003", "event_name": "add_to_cart",
  "event_time": "2025-12-26T14:29:52.652Z",
  "session_id": "ses-00001", "customer_id": 4,
  "product_id": 32, "schema_version": 1,
  "product_context": { "sku": "SKU-0032", "displayed_price": 204.22 } }
```

5.2 Índices e trade-offs

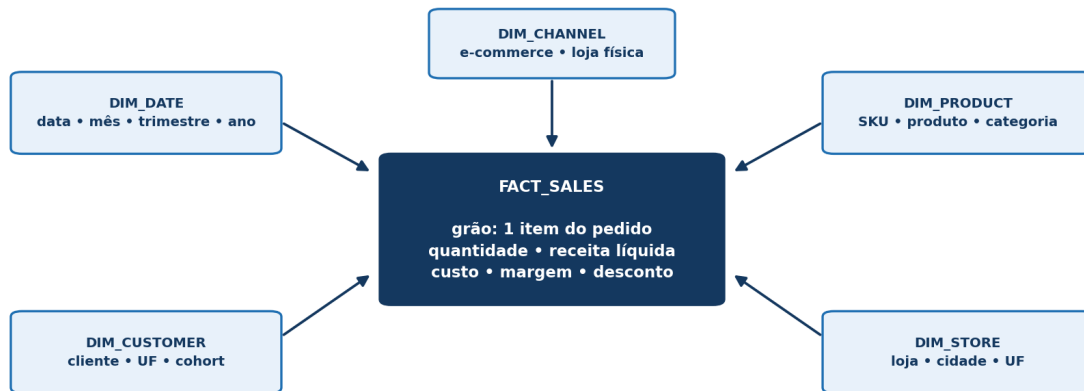
Os índices cobrem tempo, sessão, cliente, tipo e produto. A documentação do MongoDB recomenda decidir `embedding` ou referência conforme padrões de acesso, cardinalidade, frequência de atualização e crescimento do documento; `embedding` favorece leitura conjunta e atomicidade de documento, enquanto referências atendem relações amplas e ciclos independentes (MONGODB, s.d.). Cada índice aumenta I/O e memória nas escritas, portanto deve ser monitorado e removido se não sustentar consulta real.

A telemetria não substitui o pedido oficial. No VarejoMix, cada `purchase` aponta para uma venda digital confirmada com cliente, valor e quantidade compatíveis. O funil sintético tem 500 sessões: 220 com carrinho (44,0%), 123 com `checkout` (24,6%) e 78 com compra (15,6%). Essas proporções foram construídas para demonstração; não são taxas de uma empresa real. `validar_eventos.py --mongo` compara o servidor ao JSON e reconcilia os pedidos.

6. Camada analítica: dimensional e lakehouse

O modelo estrela proposto tem fact_sales no grão de item de pedido, com dimensões de data, cliente, produto, loja e canal (KIMBALL GROUP, s.d.). No recorte, analytics.vw_sales_line implementa esse grão como view, com order_item_id e line_number. As tabelas abaixo descrevem produtos Gold futuros; não são tabelas físicas já criadas.

Modelo dimensional Gold — estrela de vendas



Dimensões desnormalizadas simplificam filtros e agrupamentos; a fato guarda apenas medidas aditivas no mesmo grão.

Figura 2 - Esquema estrela proposto para vendas. Fonte: elaboração própria (2026).

PRODUTO GOLD	GRÃO	USO
fact_sales	Item de pedido válido	Receita, margem, ticket e mix por dimensões.
fact_inventory_daily	SKU × loja × dia	Ruptura, disponibilidade e cobertura.
fact_web_funnel	Sessão × etapa × dia	Conversão page_view → cart → checkout → purchase.
customer_repurchase_features	Cliente × data de corte	Propensão de recompra e segmentação reproduzível.

Dimensões podem usar chave substituta e histórico tipo 2 quando atributos precisam ser analisados “como eram” no momento da venda. Para o primeiro trimestre, cliente e produto podem começar com tipo 1 se o negócio aceitar visão atual; a evolução para tipo 2 deve ser guiada por requisitos de auditoria.

Regra única implementada em vw_valid_orders: pedidos PAID, SHIPPED ou DELIVERED com pagamento APPROVED integral. CREATED/PENDING e cancelados/reembolsados ficam fora das vendas confirmadas, cohorts e features. “Receita” neste recorte significa valor líquido dessas vendas, não apuração fiscal. Margem usa custo histórico do item; ruptura usa estoque disponível igual a zero.

7. SQL analítico, performance e integração Python

7.1 Consultas e plano de execução

sql/05_index_explain.sql compara 14 dias de pedidos DELIVERED antes e depois do índice (order_ts, status) INCLUDE (customer_id, total_amount). EXPLAIN ANALYZE executa e mede a consulta (POSTGRESQL, s.d.). Na coleta PostgreSQL 16.15 de 20/09/2026, às 20:39 UTC, ambos os planos retornaram 285 linhas: Seq Scan em 0,327 ms e Bitmap Index Scan + Heap em 0,202 ms. São os valores de Execution Time de uma única execução, sem garantia de ganho em outras condições. Fonte: evidencias/local/05_index_explain.txt.

PASSO	EVIDÊNCIA	INTERPRETAÇÃO
Sem índice	Seq Scan; 285 linhas; 0,327 ms	Varre a tabela e aplica o filtro.
Criar índice	CREATE INDEX + ANALYZE	Atualiza a estrutura e as estatísticas.
Com índice	Bitmap Index Scan + Heap; 285 linhas; 0,202 ms	Acesso seletivo observado no Docker local.
Conclusão	Execução local documentada	Não generalizar uma única medição.

7.2 Dataset de recompra

O notebook usa SQLAlchemy e pandas.read_sql para gerar uma linha por cliente com compra histórica. Corte: 01/10/2025 UTC. Features usam [corte-365 dias, corte); o alvo usa [corte, corte+90 dias). A consulta agrega pedidos separadamente das categorias, evitando duplicação por itens. A extração VarejoMix resultou em 162 clientes, dos quais 121 recompraram e 41 não recompraram (PANDAS, s.d.; SQLALCHEMY, s.d.).

FEATURE	DEFINIÇÃO / LIMITE TEMPORAL
recency_days	Dias desde a última compra antes da data de corte.
frequency_365d	Pedidos válidos nos 365 dias anteriores.
monetary_365d / average_ticket	Gasto agregado e médio somente na janela histórica.
ecommerce_share / distinct_categories	Comportamento de canal e diversidade de compra.
repurchase_90d	Alvo: compra válida após a data de corte e antes de 90 dias.

O dataset tem 162 linhas e 13 colunas, sem nulos ou IDs repetidos. O notebook tem sete células executadas sem erro salvo. A exportação de 20/09/2026 às 20:39:24 UTC corresponde ao notebook, HTML e manifesto atuais. A auditoria contém cinco registros SUCCESS e os doze snapshots preservam o mesmo hash do CSV. Os hashes dos arquivos e do código conferem com o manifesto. Limites: atividade desde dez./2024, status/categorias do snapshot e ausência de histórico completo para reconstrução temporal.

8. Benchmark de mercado e análise de trade-offs

8.1 Caso público: plataforma de dados da Uber

Como benchmark público, a Uber descreveu o DBEvents para capturar mudanças de fontes como MySQL e Cassandra, publicar changelogs em Kafka e atualizar um data lake em HDFS/Hive. O caso destaca que snapshots completos pressionavam bancos operacionais e desperdiçavam computação; a ingestão incremental melhorou freshness, qualidade e eficiência (UBER, 2019). Em outro relato, a empresa explica o uso do Apache Hudi sobre arquivos colunares para upserts, visões incrementais e uma linha do tempo de ações, evitando reescritas integrais de grandes datasets (UBER, 2020).

A VarejoMix reaproveita os princípios do caso: separar fontes operacionais e análise, capturar mudanças, usar formato colunar e preservar metadados. O caso usa Hudi; Delta Lake é uma alternativa proposta para avaliar no projeto, sem afirmar que seja automaticamente mais simples ou barata. Para o time pequeno, a implantação deve começar pelo recorte reproduzível e evoluir conforme necessidade e custo medidos.

8.2 Matriz de decisões

DECISÃO	BENEFÍCIO	CUSTO / RISCO	MITIGAÇÃO
PostgreSQL normalizado	Integridade e transações	Mais joins em leitura	Gold dimensional e índices medidos.
MongoDB por evento	Flexibilidade e escrita paralela	Consistência entre fontes	event_id, schema e reconciliação.
Parquet/Delta	Compressão, ACID e time travel	Arquivos pequenos e retenção custosa	Partição por data, compactação e VACUUM.
Batch diário primeiro	Operação simples e barata	Freshness limitada	Evoluir para CDC conforme SLA.
Desnormalização na Gold	BI simples e rápido	Duplicação controlada	Código versionado e testes de reconciliação.

8.3 Consistência versus disponibilidade

A arquitetura não aplica uma única resposta a todo o sistema. Vendas e pagamentos priorizam consistência; telemetria prioriza disponibilidade e tolera atraso; datasets Gold só ficam disponíveis após validação. Em falha parcial, a Bronze preserva o dado recebido e a Gold mantém a última versão válida, permitindo reprocessar sem interromper o consumo.

O custo de armazenamento cresce ao manter bruto e versões, mas compra auditabilidade e reprodução. A retenção deve ser explícita: versões úteis para modelos e correções permanecem; versões antigas são removidas após a janela aprovada, sem comprometer investigações ou obrigações legais.

9. Governança, implantação e conclusão

9.1 Segurança e governança

Em produção, credenciais ficam em cofre de segredos; conexões usam TLS; funções recebem menor privilégio; PII é classificada, mascarada e pseudonimizada. O dataset de ML não inclui nome ou e-mail. Catálogo, proprietário, SLA de freshness, linhagem e política de retenção acompanham cada produto de dados. Logs não devem expor payload pessoal.

9.2 Roadmap proposto de 90 dias

PERÍODO	ENTREGA	CRITÉRIO DE SAÍDA
Semanas 1-3	PostgreSQL normalizado, carga incremental e Bronze	Chaves válidas; carga repetível sem duplicação.
Semanas 4-6	Silver, Gold de vendas/estoque e painéis	Receita reconciliada; freshness diária publicada.
Semanas 7-9	Eventos MongoDB, funil e testes de qualidade	Schema versionado; conversão por etapa auditável.
Semanas 10-12	Dataset de recompra, versionamento e pitch	Features documentadas; versão fixada; execução demonstrada.

9.3 Conclusão

A VarejoMix entrega integridade relacional, eventos reconciliados e dataset auditado. A execução local registra sete verificações SQL sem erros, 11 testes aprovados e 78 compras reconciliadas. Os EXPLAIN MongoDB foram concluídos: a consulta por sessão usa IXSCAN e examina quatro documentos; a consulta de contraste usa COLLSCAN e examina 1.088 documentos. Como os filtros diferem, isso demonstra tipos de acesso, não ganho de uma mesma consulta. A exportação preserva dados e metadados reproduzíveis. A evolução proposta amplia governança, escala e reprocessamento.

Repositório: [\[PREENCHER COM O LINK DO REPOSITÓRIO\]](#)

Vídeo no YouTube: [\[PREENCHER COM O LINK DO VÍDEO\]](#)

Referências

UNIFECAP. Databases For Data Science.

APACHE PARQUET. Overview. s.d. <https://parquet.apache.org/docs/overview/>. Acesso: 5 set. 2026.

DELTA LAKE. Documentation. s.d. <https://docs.delta.io/>. Acesso: 5 set. 2026.

KIMBALL GROUP. Dimensional Modeling Techniques. s.d. <https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/dimensional-modeling-techniques/>. Acesso: 5 set. 2026.

MONGODB. Best Practices for Data Modeling. s.d. <https://www.mongodb.com/docs/manual/data-modeling/best-practices/>. Acesso: 5 set. 2026.

PANDAS. pandas.read_sql. s.d. https://pandas.pydata.org/docs/reference/api/pandas.read_sql.html. Acesso: 5 set. 2026.

POSTGRESQL. Using EXPLAIN - versão 16. s.d. <https://www.postgresql.org/docs/16/using-explain.html>. Acesso: 5 set. 2026.

SQLALCHEMY. Establishing Connectivity - the Engine. s.d. <https://docs.sqlalchemy.org/en/20/tutorial/engine.html>. Acesso: 5 set. 2026.

UBER. DBEvents: A Standardized Framework for Efficiently Ingesting Data into Uber's Apache Hadoop Data Lake. 2019. <https://www.uber.com/us/en/blog/dbevents-ingestion-framework/>. Acesso: 5 set. 2026.

UBER. Building a Large-scale Transactional Data Lake at Uber Using Apache Hudi. 2020. <https://www.uber.com/us/en/blog/apache-hudi-graduation/>. Acesso: 5 set. 2026.